



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
RAI TECHNOLOGY UNIVERSITY

Estd. under Karnataka Act. No. 40 of 2013

Assignment No: 02

Subject Title: Deep Learning

Subject Code: 0CS351T

Submitted By:

Student Name: D Suresh

University Enrolment Number: RTU24101IT019

Program Name: B. TECH IT AIML

Semester: Vth Sem

Session: 2026-2027

Submitted To:

Faculty Name: Mr Kumar Satyam Tanti

College Of Engineering & Computer Application

Rai Technology University

ASSIGNMENT - 02

MNIST HANDWRITTEN DIGIT CLASSIFICATION

1) INTRODUCTION: Handwritten digit recognition is a classic problem in deep learning where a model learns to identify which digit (0-9) is present in an image, based purely on pixel patterns rather than handed-coded rules. This assignment uses the MNIST dataset - a collection of 70,000 grayscale images of handwritten digits - to build, train, & evaluate a simple feedforward neural network using TensorFlow / Keras. The task is treated as a multi-class classification problem, where each input image must be mapped to one of ten possible output classes.

2) OBJECTIVE:

- To load & explore the MNIST dataset & visualize sample images.
- To preprocess the data (normalization & flattening) so it can be fed into a neural network.
- To design, compile, & train a neural network for digit classification.
- To evaluate the trained model's accuracy on unseen test data.

- To visualize training vs validation accuracy & loss to check for overfitting.
- To test the model on individual images & compare predicted vs actual labels.
- To conduct one controlled experiment (adding Dropout) & compare its effect against the baseline model.

3) **THEORY:** A neural network is a system of layers of interconnected neurons that learns to map inputs to outputs by adjusting internal weights during training. For image classification, a basic feedforward network - also called a Multi-Layer Perceptron (MLP) - works as follows:

- **Input layer:** Receives the flattened pixel values of an image (784 values for a 28×28 image).
- **Hidden layer(s):** Apply weighted sums followed by an activation fn. allowing the network to learn non-linear combinations of the input features.
- **Output layer:** Produces a score (or probability) for each of the 10 digit classes.

Training works by comparing the network's predictions to the true labels using a loss function, then adjusting weights via backpropagation & gradient descent to reduce that loss. The Adam optimizer was used in this - it adapts the learning rate for each parameter automatically, which generally leads to faster, more stable convergence than plain Stochastic Gradient Descent (SGD).

The loss fn. used was Sparse Categorical Crossentropy, the correct choice for multi-class classification when labels are given as plain integers (0-9) rather than one-hot encoded vectors.

4) ACTIVATION FUNCTIONS

i) ReLU (Rectified Linear Unit) - used in the hidden layer.

$$\rightarrow f(x) = \max(0, x)$$

→ Outputs the input directly if positive, otherwise outputs zero.

→ Introduces non-linearity cheaply & avoids the vanishing gradient problem that older activations (like sigmoid) suffer from in deeper networks.

ii) Sigmoid - used in the output layer of Model Attempts 1 & 2.

→ Formula: $p(x) = 1/(1 + e^{-x})$

→ Squashes values into the range $(0, 1)$, but does not guarantee that the outputs across all 10 classes sum to 1.

iii) Softmax - used in the output layer of the final baseline & the experiment model.

→ Converts the 10 raw output scores into a proper probability distribution that sums to 1 across all classes.

→ This is the technically correct activation for multi-class classification, which is why it was adopted for the final baseline & experiment models.

5) DATASET :

→ Name: MNIST Handwritten Digits Dataset.

→ Total images: 70,000 (60,000 training / 10,000 test)

→ Image size: 28×28 pixels, grayscale

→ Pixel range: 0-255, normalized to 0-1

→ Classes: 10 (digits 0-9), roughly balanced

→ Source: Loaded directly via

`keras.datasets.mnist.load_data()`

6) Model: Two models were built & compared, followed by a final baseline used for the rest of the assignment:

Model Attempt 1 (no hidden layer): Epochs: 5; Optimizer - Adam
Dense(10, activation = 'sigmoid')

Model Attempt 2 (with hidden layer): Epochs: 5; Optimizer - Adam
Dense(100, activation = 'relu') → Dense(10, activation = 'sigmoid')

Final Baseline: Dense(100, activation = 'relu')

→ Dense(10, activation = 'softmax'), trained with a 10% validation split. Epochs = 10; Optimizer - Adam.

7) CODE:

Importing Libraries:

import tensorflow as tf

from tensorflow import keras

import matplotlib.pyplot as plt

%matplotlib inline

import numpy as np.

Loading Dataset:

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.

len(x_train)

len(x_test)

load_data()

len(x_train)

60000

10000

len(x_test)

x_train[0].shape

x_train[0].shape

(28, 28)

文、理、工、农、医(卷)

array (28, 28) node data

[illegible][illegible]

```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [18, 18, 18, 126, 136, 175, 26, 166, 255, 247, 127, 0, 0],
       [0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 30, 36, 94, 154, 179,
        253, 253, 253, 253, 253, 225, 172, 253, 242, 195, 64, 0, 0],
       [0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 40, 238, 253, 253, 253, 253,
        253, 253, 253, 251, 93, 82, 82, 56, 39, 0, 0, 0],
       [0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 18, 219, 253, 253, 253, 253,
        253, 198, 182, 247, 241, 0, 0, 0, 0, 0, 0, 0],
       [0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 80, 156, 107, 253, 253,
        205, 11, 0, 43, 154, 0, 0, 0, 0, 0, 0, 0],
       [0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 14, 1, 154, 253,
        90, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0]
```

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

Sample MNIST Training Images

[illegible][illegible][illegible][illegible]

```
x_train_flattend = x_train.reshape(len(x_train), 28*28)
```

```
x_test_flattend = x_test.reshape(len(x_test), 28*28)
```

```
x_train_flattend.shape
```

x_train_flattend.shape

(60000, 784)

```
x_test_flattend.shape
```

```
x_test_flattend[0]
```

x_test_flattend.shape

(10000, 784)

```
# Model 1 - No hidden layer
```

```
model = keras.Sequential([
```

```
keras.layers.Dense(10, input_shape=(784), activation =  
    'sigmoid')])
```

```
model.compile(
```

```
optimizer='adam', loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])
```

```
model.fit(x_train_flattend, y_train, epochs=5)
```

Epoch 1/5

1875/1875 — 11s 5ms/step - accuracy: 0.8772 - loss: 0.4704

Epoch 2/5

1875/1875 — 3s 2ms/step - accuracy: 0.9152 - loss: 0.3045

Epoch 3/5

1875/1875 — 3s 2ms/step - accuracy: 0.9201 - loss: 0.2837

Epoch 4/5

1875/1875 — 4s 2ms/step - accuracy: 0.9234 - loss: 0.2732

Epoch 5/5

1875/1875 — 4s 2ms/step - accuracy: 0.9251 - loss: 0.2669

```
model.evaluate(x_test_flattend, y_test)
```

```
plt.matshow(x_test[0])
```

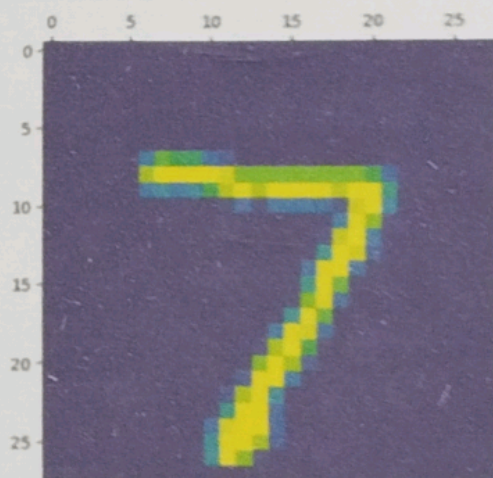
```
y_predicted = model.predict  
                (x_test_flattend)
```

```
y_predicted[0]
```

```
np.argmax(y_predicted[0])
```

```
y_predicted_labels = [np.argmax(i)  
    for i in y_predicted]
```

```
y_predicted_labels[:5]
```



y_test[:5]

y_predicted[0]

313/313 — 0s 1ms/step

cm = tf.math.confusion-
matrix(labels=y_test,
predictions=y_predicted_labels)

array([[1.42978800e-02, 4.78070149e-07, 1.82573216e-01, 9.68940620e-01,
4.00834577e-03, 1.38741583e-01, 3.09966117e-06, 9.90802295e-01,
1.56533927e-01, 6.87755108e-01], dtype=float32)

cm

np.argmax(y_predicted[0])

import seaborn as sn

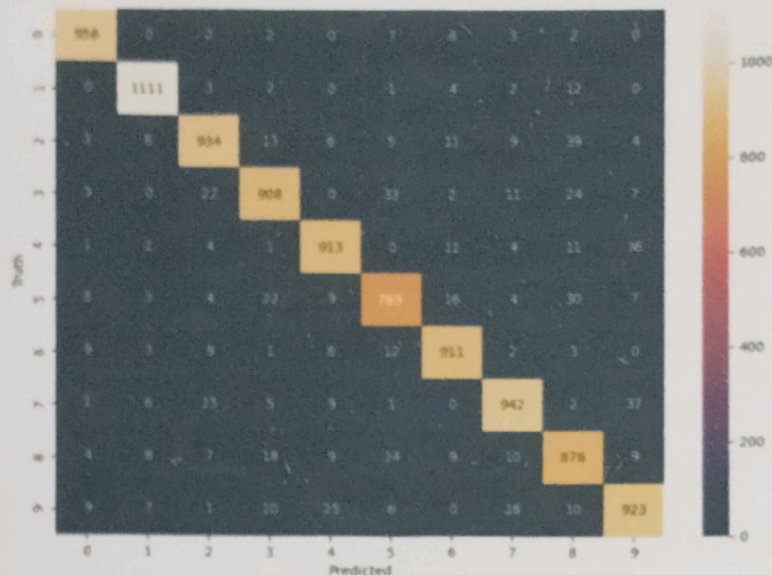
np.int64(7)

plt.figure(figsize=(10,7))

sn.heatmap(cm, annot=True, fmt='d')

plt.xlabel('Predicted')

plt.ylabel('Truth')



Model-2 - with hidden layer.

model = keras.Sequential([

keras.layers.Dense(100, input_shape=(784,), activation='relu'),

keras.layers.Dense(10, activation='sigmoid')])

model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_flattened, y_train, epochs=5)

Epoch 1/5
1875/1875 — 7s 4ms/step - accuracy: 0.9216 - loss: 0.2741
Epoch 2/5
1875/1875 — 5s 3ms/step - accuracy: 0.9631 - loss: 0.1263
Epoch 3/5
1875/1875 — 6s 3ms/step - accuracy: 0.9731 - loss: 0.0894
Epoch 4/5
1875/1875 — 9s 3ms/step - accuracy: 0.9800 - loss: 0.0668
Epoch 5/5
1875/1875 — 7s 4ms/step - accuracy: 0.9835 - loss: 0.0538

`model.evaluate(x_test_flattend, y_test)`

`y_predicted = model.predict(x_test_flattend)`

`y_predicted_labels = [np.argmax(i) for i in y_predicted]`

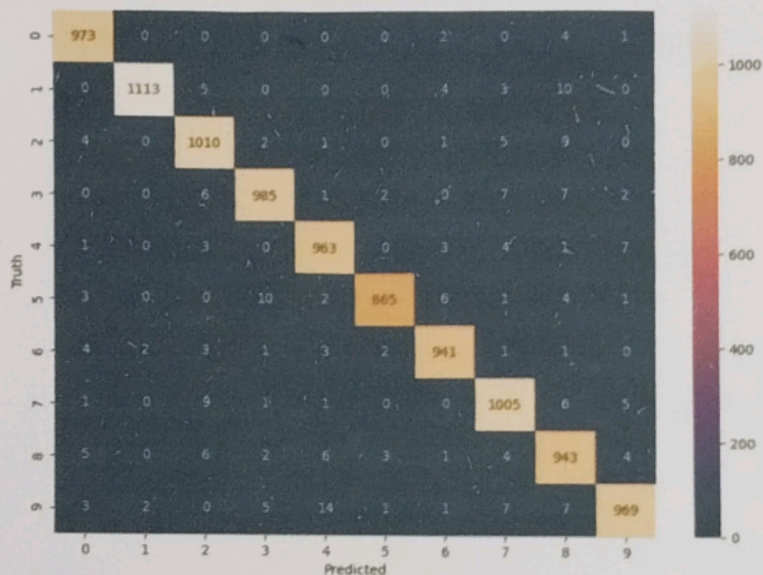
`cm = tf.math.confusion_matrix(labels=y_test, predictions
= y_predicted_labels)`

`plt.figure(figsize=(10,7))`

`sn.heatmap(cm, annot=True, fmt='d')`

`plt.xlabel('Predicted')`

`plt.ylabel('Truth')`



Baseline with a Validation Split

`baseline_model = keras.Sequential([`

`keras.layers.Dense(100, input_shape=(784,)),`

```

activation = 'relu'),
keras.layers.Dense(10, activation = 'softmax')])
baseline_model.compile(
    optimizer = 'adam', loss = 'sparse_categorical_crossentropy',
    metrics = ['accuracy'])
history = baseline_model.fit(
    x_train_flattened, y_train, epochs = 10, validation_split = 0.1,
    verbose = 1
)

```

```

baseline_test_loss, baseline_test_accuracy = baseline_model.
    evaluate(x_test_flattened, y_test, verbose = 0)

```

```

print(f"Baseline Test Accuracy: {baseline_test_accuracy:.4f}
      | Test Loss: {baseline_test_loss:.4f}")

```

```

Epoch 1/10 6s 3ms/step - accuracy: 0.9173 - loss: 0.2897 - val_accuracy: 0.9630 - val_loss: 0.1377
Epoch 2/10 8s 3ms/step - accuracy: 0.9608 - loss: 0.1317 - val_accuracy: 0.9730 - val_loss: 0.1090
Epoch 3/10 6s 3ms/step - accuracy: 0.9724 - loss: 0.0925 - val_accuracy: 0.9703 - val_loss: 0.1025
Epoch 4/10 6s 4ms/step - accuracy: 0.9784 - loss: 0.0716 - val_accuracy: 0.9723 - val_loss: 0.0966
Epoch 5/10 5s 3ms/step - accuracy: 0.9833 - loss: 0.0563 - val_accuracy: 0.9745 - val_loss: 0.0850
Epoch 6/10 7s 4ms/step - accuracy: 0.9862 - loss: 0.0442 - val_accuracy: 0.9772 - val_loss: 0.0814
Epoch 7/10 5s 3ms/step - accuracy: 0.9885 - loss: 0.0367 - val_accuracy: 0.9792 - val_loss: 0.0780
Epoch 8/10 7s 4ms/step - accuracy: 0.9906 - loss: 0.0304 - val_accuracy: 0.9802 - val_loss: 0.0735
Epoch 9/10 8s 3ms/step - accuracy: 0.9923 - loss: 0.0251 - val_accuracy: 0.9782 - val_loss: 0.0855
Epoch 10/10 10s 3ms/step - accuracy: 0.9934 - loss: 0.0210 - val_accuracy: 0.9790 - val_loss: 0.0799
Baseline Test Accuracy: 0.9754 | Test Loss: 0.0856

```

Visualisation

```

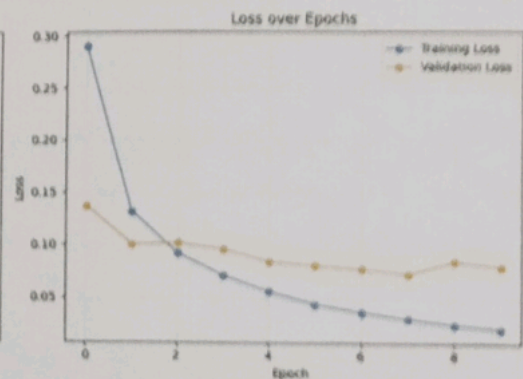
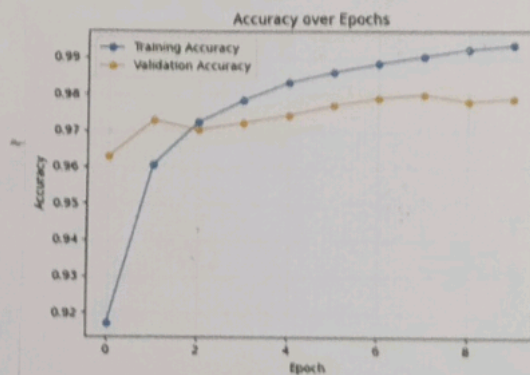
fig, axes = plt.subplots(1, 2, figsize = (12, 4.5))
axes[0].plot(history.history['accuracy'], label = 'Training
    Accuracy', marker = 'o')
axes[0].set_title('Accuracy over Epochs')

```

```

axes[0].plot(history.history['val_accuracy'], label =
              'Validation Accuracy', markers = 'o')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(alpha = 0.3)
axes[1].plot(history.history['loss'], label = 'Training
              loss', marker = 'o')
axes[1].plot(history.history['val_loss'], label = 'Validation
              loss', marker = 'o')
axes[1].set_title('Loss over Epochs')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(alpha = 0.3)
plt.tight_layout()
plt.savefig('training_curves.png', dpi = 150, bbox_inches
           = 'tight')
plt.show()

```



```

# Testing model on five handwritten images
np.random.seed(7)
sample_indices = np.random.choice(len(x_test_flattened),
                                   5, replace=False)
sample_predictions = baseline_model.predict(x_test_flattened[
    sample_indices], verbose=0)
predicted_labels_sample = np.argmax(sample_predictions,
                                     axis=1)

actual_labels_sample = y_test[sample_indices]
fig, axes = plt.subplots(1, 5, figsize=(14, 3))
for i, idx in enumerate(sample_indices):
    axes[i].imshow(x_test[idx], cmap='gray')
    correct = predicted_labels_sample[i] == actual_labels_sample[i]
    color = 'green' if correct else 'red'
    axes[i].set_title(f"Actual: {actual_labels_sample[i]}  

    \n Predicted: {predicted_labels_sample[i]}, color={color}")
    axes[i].axis('off')
plt.suptitle("Model Predictions on 5 Test Images (Actual vs Predicted)")
plt.tight_layout()
plt.savefig('predictions_sample.png', dpi=150, bbox_inches='tight')
plt.show()
for idx, a, p in zip(sample_indices, actual_labels_sample,
                     predicted_labels_sample):

```

print(f"Image index {idx}: Actual = {a}, Predicted = {p}
 → 'correct' if a == p else 'incorrect'")

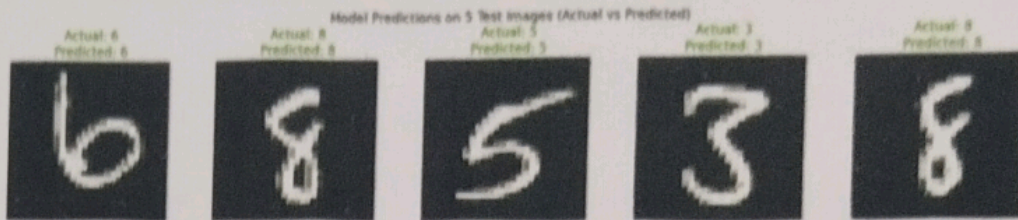


Image index 1977: Actual = 6, Predicted = 6 → Correct
 Image index 3880: Actual = 8, Predicted = 8 → Correct
 Image index 52: Actual = 5, Predicted = 5 → Correct
 Image index 2551: Actual = 3, Predicted = 3 → Correct
 Image index 2246: Actual = 8, Predicted = 8 → Correct

Experiment - Dropout

```
experiment_model = keras.Sequential([
    keras.layers.Dense(100, input_shape=(784),
                        activation='relu'),
    keras.layers.Dense(10, activation='softmax')
    keras.layers.Dense(0.3),
```

1)

```
experiment_model.compile(optimizer='adam',
                        loss='sparse_categorical_crossentropy',
                        metrics=['accuracy'])
```

)

```
history_experiment = experiment_model.fit(
    x_train_flattend, y_train, epochs=10,
    validation_split=0.1, verbose=1
```

)

```
exp_test_loss, exp_test_accuracy = experiment_model.
    evaluate(x_test_flattend, y_test, verbose=0)
```

```
print(f"Experiment (Dropout) Test Accuracy : {exp_test_
```

accuracy: 48% | Test loss: exp-test-loss: 48%

```
Epoch 1/10
1688/1688 8s 4ms/step - accuracy: 0.8934 - loss: 0.3646 - val_accuracy: 0.9635 - val_loss: 0.1344
Epoch 2/10
1688/1688 5s 3ms/step - accuracy: 0.9439 - loss: 0.1926 - val_accuracy: 0.9703 - val_loss: 0.1053
Epoch 3/10
1688/1688 7s 4ms/step - accuracy: 0.9538 - loss: 0.1526 - val_accuracy: 0.9745 - val_loss: 0.0863
Epoch 4/10
1688/1688 5s 3ms/step - accuracy: 0.9600 - loss: 0.1296 - val_accuracy: 0.9777 - val_loss: 0.0806
Epoch 5/10
1688/1688 7s 4ms/step - accuracy: 0.9653 - loss: 0.1140 - val_accuracy: 0.9785 - val_loss: 0.0776
Epoch 6/10
1688/1688 9s 3ms/step - accuracy: 0.9688 - loss: 0.1016 - val_accuracy: 0.9778 - val_loss: 0.0793
Epoch 7/10
1688/1688 6s 4ms/step - accuracy: 0.9706 - loss: 0.0944 - val_accuracy: 0.9803 - val_loss: 0.0699
Epoch 8/10
1688/1688 5s 3ms/step - accuracy: 0.9717 - loss: 0.0875 - val_accuracy: 0.9792 - val_loss: 0.0732
Epoch 9/10
1688/1688 7s 4ms/step - accuracy: 0.9742 - loss: 0.0804 - val_accuracy: 0.9800 - val_loss: 0.0693
Epoch 10/10
1688/1688 10s 4ms/step - accuracy: 0.9758 - loss: 0.0774 - val_accuracy: 0.9803 - val_loss: 0.0750
Experiment (Dropout) Test Accuracy: 0.9782 | Test Loss: 0.0782
```

Comparison b/w Baseline & Dropout

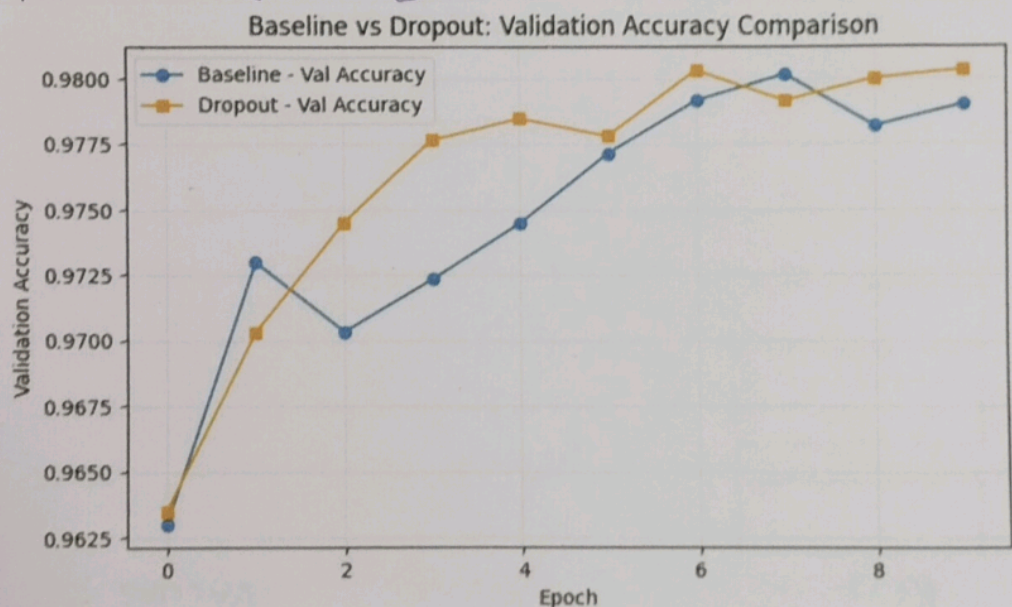
```
print(f"{'Model': <28}{'Test Accuracy': <18}{'Test Loss': <12}")
print("-" * 58)
```

```
print(f"{'Baseline (no dropout)': <28}{'baseline-test-accuracy': <18.48}{'baseline-test-loss': <12.48}")
```

```
print(f"{'Experiment (dropout)': <28}{'exp-test-accuracy': <18.48}{'exp-test-loss': <12.48}")
```

```
plt.figure(figsize = (8, 4.5))
```

```
plt.plot(history.history['val_accuracy'], label = 'Baseline -
```



```

-Val Accuracy', marker='o')
plt.plot(history - experiment.history['val accuracy'],
         label='Dropout - Val Accuracy', marker='s')
plt.title('Baseline vs Dropout: Validation Accuracy Comparison')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.grid(alpha=0.3)
plt.savefig('baseline-vs-experiment.png', dpi=150,
          bbox_inches='tight')
plt.show()

```

8) RESULT:

Model	Test Accuracy	Test Loss
Model 1 - no hidden layer	92.63%	0.2647
Model 2 - hidden layer, sigmoid o/p	97.67%	0.0782
Baseline (softmax, validation split)	97.54%	0.0856

The training/validation curves for the final baseline showed training accuracy climbing to 99.3% by epoch 10 while validation accuracy stayed around 97.9-98%, indicating a small amount of overfitting - the model performs slightly better on training data than on unseen validation data.

9) Experiment: Added Dropout (0.5) after the hidden layer, keeping the architecture, optimizer, loss, epochs, & validation split identical to the final baseline - isolating the effect of dropout alone.

Model	Test Accuracy	Test Loss
Baseline (no dropout)	97.54%	0.0856
Experiment (with dropout)	97.82%	0.0782

Observation: The dropout model achieved both higher test accuracy & lower test loss than the baseline. More importantly, in the baseline the final-epoch training accuracy (99.3%) was noticeably higher than its validation accuracy (97.9%), a 1.4% gap indicating mild overfitting. In the dropout model, training accuracy (97.6%) & validation accuracy 98.0% stayed much closer together - dropout successfully reduced overfitting by preventing the network from relying too heavily on specific neurons, which in this case also translated into slightly better generalization on the test set.

10) CONCLUSION: This assignment covered the complete deep learning workflow for image classification: loading & exploring data, preprocessing, building & training multiple model variants,

evaluating performance, visualizing learning curves, testing individual predictions, & running a controlled experiment.

→ Adding a hidden layer improved accuracy substantially (92.63% → 97.67%), confirming that non-linear feature learning matters even for a "simple" dataset like MNIST.

→ Switching to Softmax output is the theoretically correct choice for multi-class problems, even though Sigmoid also worked reasonably well here.

→ Adding Dropout (0.3) improved both test accuracy (97.54% → 97.82%) & generalization, demonstrating the practical value of regularization.